



UCD School of Physics

PHYC40600 Physics Astronomy and Space Lab II

Newton's Cooling

Joana C.C. Adao

Student No.: 23311051

22nd September 2026

Abstract

This experiment investigated an aluminium rod heated at one end with lateral Newtonian losses acting on it, to test the temperature profile approaching a steady-state and Prigogine's nonequilibrium minimum entropy theorem. Sixteen calibrated thermocouples recorded the temperature distribution over a 75 minute measurement window with different initial conditions (uniform vs. non-uniform, locally pre-heated). The steady-state profile in both runs were well described by an exponential decay, yielding a decay constant of $\beta = 2.8216 \pm 0.0264 \text{ m}^{-1}$ (uniform) and $2.8042 \pm 0.0261 \text{ m}^{-1}$ (non-uniform), consistent within uncertainty despite the differing initial conditions. The entropy production rate $d_i S/dt$ remained non-negative throughout the runs and decreased monotonically to a non-zero steady-state minimum (0.2086 ± 0.0091 and 0.2186 ± 0.009), confirming both the second law and Prigogine's theorem, though $-d_e S/dt$ crossed and slightly exceeded $d_i S/dt$ by the end of the run rather than completely converging onto it exactly.

Contents

1	Introduction	1
1.1	Heat Conduction & Lateral Newtonian Losses	1
1.2	Nonequilibrium Thermodynamics	1
1.3	Minimum Entropy Production	2
2	Methodology	2
2.1	NI Setup	2
2.2	Calibration & Data Acquisition	2
2.3	Data Analysis	3
2.3.1	Effective Oven Temperature	3
2.3.2	Steady-State Temperature Profile	3
2.3.3	Entropy Transfer and Production	3
3	Results	4
3.1	Calibration	4
3.2	Effective Oven Temperature	4
3.3	Steady-State Temperature Profile	4
3.4	Entropy Transfer and Production	4
4	Analysis and Discussion	5
5	Conclusion	6
	Appendix	9
	Tables	9
	Code	9

1. Introduction

Classic thermodynamics often assumes and describes systems in equilibrium, where temperature and pressure variables are typically uniform and time-independent. However, real systems are almost never in true equilibrium. Instead, they are driven by continuous fluxes in energy or matter until they settle into a nonequilibrium steady state. The way systems approach and behave in these states will be explored. [1, 2]

1.1. Heat Conduction & Lateral Newtonian Losses

The conservation of heat in a solid, following Fourier's law of heat conduction $\mathbf{J}_q = -\lambda \nabla T$, gives the general differential equation describing the heat distribution in the rod [2, 3]:

$$\rho c \frac{dT}{dt} = \nabla \cdot (\lambda \nabla T) \quad (1)$$

with ρ as the density, c as the specific heat, and λ as the thermal conductivity of the rod. For a rod losing heat through its lateral surface to an external bath of temperature T_R , Newton's law of cooling is implemented as the boundary condition [2, 3]:

$$\lambda \hat{\rho}_0 \cdot (\nabla T) + H(T - T_R) = 0 \quad (2)$$

This states that the normal heat flow through the surface of the rod is proportional to the difference in temperature with respect to the external bath. Here, H is the surface conduction, and $\hat{\rho}_0$ is the outward radial (normal) unit vector. [2, 3]

When the rod's thermal conductivity is high relative to its radius, the radial temperature gradients remain small relative to its axial variation. Averaging across the rod's cross-section then folds this boundary condition into an effective source term within a purely one-dimensional equation [2, 3]:

$$\rho c \frac{\partial T(z, t)}{\partial t} = \lambda \frac{\partial^2 T(z, t)}{\partial z^2} - \frac{2H}{r} [T(z, t) - T_R] \quad (3)$$

with r as the rod's radius. This equation interpolates between the two limits of the boundary condition discussed for the rod: for $H \rightarrow 0$ representing a perfectly insulating surface, and $H \rightarrow \infty$ representing a perfectly conductive surface. [2]

Under a fixed boundary temperature difference, the rod cannot relax to equilibrium. Instead,

it approaches a time-independent, nonequilibrium steady state given as $T_{SS}(z)$, which is found by setting $\partial T / \partial t = 0$ [2]:

$$[T_{SS}(z, \infty) - T_R] = [T_L - T_R] e^{-\beta z} \quad (4)$$

with T_L as the extrapolated boundary temperature at $z = 0$, and the coefficient β given by $\beta = \sqrt{2H/\lambda r}$. As β depends only on the thermal and geometric properties of the rod and its surroundings (H, λ, r), it is treated as a constant that can be purely experimentally determined from the steady-state temperature distribution. [2, 3]

1.2. Nonequilibrium Thermodynamics

In thermodynamics, entropy is defined only for states in thermodynamic equilibrium. Therefore, to extend this to a nonequilibrium system, the local equilibrium hypothesis is considered: the system is subdivided into volume elements that are each large enough to have well-defined temperatures and pressures, yet small enough that these values do not vary appreciably within the element. Gibbs' relation is assumed to hold for each element [2]:

$$ds = \frac{1}{T} du \quad (5)$$

with s and u as the entropy and internal energy, respectively. When combined with the local energy balance it yields the local entropy balance equation [1, 2]:

$$\rho \frac{ds}{dt} + \nabla \cdot \mathbf{J}_s = \sigma \quad (6)$$

with $\mathbf{J}_s = \mathbf{J}_q / T$ as the entropy flux density, and $\sigma = \mathbf{J}_q \cdot \nabla (1/T)$ as the local entropy production. The second law of thermodynamics requires $\sigma \geq 0$ at every point. [1, 2]

Integrating over the whole rod separates the entropy change into two physically distinct contributions [1, 2]:

$$\frac{dS}{dt} = \frac{d_e S}{dt} + \frac{d_i S}{dt} \quad (7)$$

with $d_e S / dt$ as the entropy exchanged with the surroundings across the system's boundary, and $d_i S / dt$ as the exchanges inside the system. The second law thus requires that $d_i S / dt \geq 0$ at all times, with equality only possible for reversible processes. [2, 4]

1.3. Minimum Entropy Production

Near equilibrium, the thermodynamic heat flux J relates linearly to its conjugate force $X = \nabla(1/T)$ by $J = LX$. When several fluxes and forces are present, the Onsager reciprocal relations $L_{ij} = L_{ji}$ govern the coupling between them. [1, 5] Within this linear regime, Prigogine's theorem of minimum entropy production states that a system relaxing toward a nonequilibrium steady-state under fixed constraints does so by monotonically decreasing its entropy production rate, which reaches a generally non-zero minimum value corresponding to the steady state. [1, 4]

The distinction between fixed and free thermodynamic forces is critical to the theorem. The steady-state is not defined by a zero total entropy production, but rather by the vanishing of the fluxes conjugate to the free forces. [1]

2. Methodology

The experimental apparatus is shown in Figure 1.

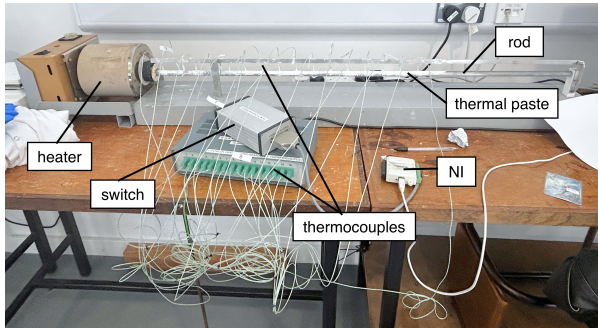


Figure 1: Experimental setup with equipment labels.

One end of an aluminium rod was inserted into a furnace, while the opposite end extended far enough to remain at ambient (room) temperature, acting as the external bath. Sixteen thermocouples were placed along the rod at a uniform spacing of 5 cm, beginning at the furnace end. Thermal paste was applied both to the thermocouple notches and to the exposed wire ends to improve thermal contact and ensure accurate readings. The thermocouples were connected to an 8-way switching box, which was in turn connected to a National Instruments USB-6008 data acquisition device.

2.1. NI Setup

Since the USB-6008 provides only eight analogue input channels while the experiment required sixteen thermocouple readings, the switching box was used to measure and stack two sets of eight channels onto a single set of inputs. Acquisition code was written to accommodate this. The channels were configured as single-ended (rather than the default differential mode) [6]:

```
1 myADC = ADC()
2 myDAC = DAC(0)
3
4 myADC.addChannels([0,1,2,3,4,5,6,7],
5                   ADC_mode='DAQmx_Val_RSE')
```

Sixteen readings were then acquired by toggling the switching box between its two states, with 0 V selecting thermocouples 1-8, and 5 V selecting thermocouples 9-16.

```
1 def thermocouple_16loop():
2     voltages=np.zeros(16)
3
4     myDAC.writeVoltage(0)
5     time.sleep(0.05)
6     set1=myADC.sampleVoltages(1)
7     for i in range(8):
8         voltages[i]=set1[i][0]
9
10    myDAC.writeVoltage(5)
11    time.sleep(0.05)
12    set2=myADC.sampleVoltages(1)
13    for i in range(8):
14        voltages[i+8]=set2[i][0]
15
16    myDAC.writeVoltage(0)
17    return voltages
```

The function first creates an empty 16-element array, then records the first eight thermocouple voltages with the switch at 0 V, allowing 0.05 s for the switch to settle before each read, before repeating the process for the remaining eight thermocouples with the switch at 5 V. The switch is returned to 0 V at the end of each run in preparation for the next set of measurements.

2.2. Calibration & Data Acquisition

To convert the raw NI voltage readings into temperatures, each thermocouple was calibrated individually. The sixteen thermocouples were bundled together and immersed in a beaker of water heated by kettle. The water's temperature was measured directly with a thermometer, while the corresponding thermocouple voltages were recorded using the acquisition loop defined. Paired thermometer-voltage readings were then taken continuously as the water cooled to

room temperature. A linear fit of temperature against voltage, $T = V \cdot m + c$, was performed for each thermocouple individually, and the resulting slope m and intercept c were used to convert all subsequent voltage measurements from that thermocouple into temperature.

Two runs were then recorded. In the uniform run, the rod began at room temperature before the furnace was switched on. In the non-uniform run, the centre of the rod was first heated locally with a hairdryer before the furnace was switched on. Both runs lasted 75 minutes, with all sixteen thermocouples read and logged once every 60 seconds.

2.3. Data Analysis

Three measurements were calculated from the calibrated temperature data: the effective oven temperature, the steady-state temperature profile, and the rates of entropy transfer and production.

2.3.1 Effective Oven Temperature

The furnace's dial markings do not reliably indicate the true boundary temperature seen by the rod, since it does not account for ambient losses, contact resistance, or insulation effects. The true value T_L was instead obtained from the measured steady-state profile using Equation (4), rearranged by taking the natural logarithm of both sides:

$$\ln[T(z) - T_R] = \ln(T_L - T_R) - \beta z \quad (8)$$

A plot of $\ln[T(z) - T_R]$ against the thermocouple positions z is therefore linear, with a slope of $-\beta$ and intercept $\ln(T_L - T_R)$. T_L was determined by extrapolating this linear fit to $z = 0$, rather than reading directly from any single thermocouple as none was located exactly at the furnace boundary.

2.3.2 Steady-State Temperature Profile

The full time evolution of the temperature distribution was visualized with a 3D waterfall plot, with axes of time t , position z , and temperature T , each curve corresponding to one thermocouple. This used the calibrated temperature data directly, requiring no further calculation.

2.3.3 Entropy Transfer and Production

The rates of entropy transfer across the rod's boundary ($d_e S/dt$) and entropy production within

it ($d_i S/dt$), were evaluated using expressions written in terms of experimentally accessible quantities:

$$\begin{aligned} \frac{d_e S}{dt} \propto & \int_0^L \frac{1}{T} \frac{\partial^2 T}{\partial z^2} dz - \int_0^L \frac{1}{T^2} \left(\frac{\partial T}{\partial z} \right)^2 dz \\ & - \frac{2H}{\lambda r} \int_0^L \frac{1}{T} (T - T_R) dz \end{aligned} \quad (9)$$

$$\frac{d_i S}{dt} \propto \int_0^L \frac{1}{T^2} \left(\frac{\partial T}{\partial z} \right)^2 dz \quad (10)$$

To avoid the numerical instability of computing a second spatial derivative due to the low density of experimental data, a discrete version of Equations (9) and (10) was used utilising experimental values: the measured thermocouple temperatures T_i , thermocouple spacing l , and the fitted decay constant β :

$$\begin{aligned} \frac{d_e S}{dt} \propto & -\frac{1}{l} \frac{1}{T_0} (T_1 - T_0) + \frac{1}{l} \frac{1}{T_n} (T_n - T_{n-1}) \\ & - \beta^2 l \sum_{i=1}^{n-1} \left(1 - \frac{T_n}{T_i} \right) \end{aligned} \quad (11)$$

$$\frac{d_i S}{dt} \propto -\frac{1}{l} \sum_{i=0}^{n-1} (T_{i+1} - T_i) \left(\frac{1}{T_{i+1}} - \frac{1}{T_i} \right) \quad (12)$$

Here, $T_0 = T_L$ and $T_n = T_R$ denote the two fixed boundary temperatures, and all temperatures were expressed in Kelvin, since the $1/T$ terms require an absolute temperature scale. Results are presented as $d_i S/dt$ and $-d_e S/dt$, following the convention that makes both quantities converge to the same value and hence directly comparable to the steady-state with an established entropy minimum.

In practice, TCouple 16 had not fully relaxed to T_R by the end of the either run, remaining several degrees above the room temperature the thermocouples were calibrated to. Since Equations (11) and (12) assume the last measurement already sits at ambient boundary temperature, applying the measured T_R readings would create a very large residual gap between $d_i S/dt$ and $d_e S/dt$. To correct for this, "virtual thermocouples" were extended beyond TCouple 16 following the same exponential profile of Equation (4), with the same spacing z , until the modelled temperature fell within a small tolerance of T_R .

3. Results

3.1. Calibration

The calibration data for all sixteen thermocouples taken for seven reference temperatures, from room temperature (20.2 °C) to kettle-boiled temperature (76.5 °C), are shown in Figure 2 alongside their individual fits. All sixteen curve fits are visually linear over the full measured voltage range.

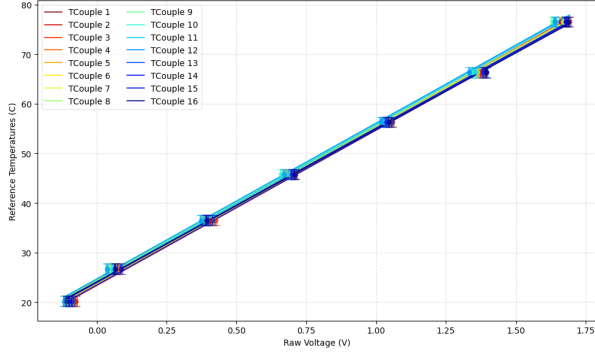


Figure 2: Calibration plot.

Each thermocouple was individually fitted to $T = V \cdot m + c$ by least squares regression, using the manufacturer-specified thermometer 1 °C uncertainty. The resulting uncertainty on each fitted slope σ_m and intercept σ_c , together with their covariance $\text{Cov}(m, c)$, were then propagated onto the uncertainty on the measured temperatures with:

$$\sigma_T^2 = V^2 \sigma_m^2 + \sigma_c^2 + 2V \text{Cov}(m, c) \quad (13)$$

3.2. Effective Oven Temperature

Figure 3 shows $\ln[T(z) - T_R]$ plotted against the thermocouple positions (z) for both the uniform and non-uniform runs. Both sets of data were fitted with a straight line over the full 0.05-0.80 m range, consistent with the exponential steady-state profile described in Equation (4).

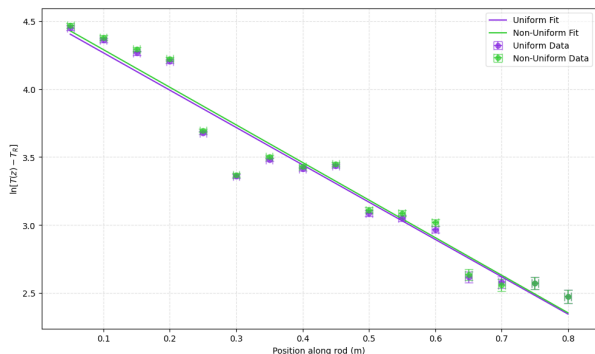


Figure 3: Effective oven temperature plot.

Extrapolating each linear fit to $z=0$ gives the values of:

$$\begin{aligned} \beta_{\text{unif}} &= 2.7458 \text{ m}^{-1} & \beta_{\text{non-unif}} &= 2.7656 \text{ m}^{-1} \\ T_{L,\text{unif}} &= 113.99 \text{ }^\circ\text{C} & T_{L,\text{non-unif}} &= 116.39 \text{ }^\circ\text{C} \end{aligned}$$

The uncertainties on β and T_L were determined using the corresponding weighted fit of Equation (8). Per-point weighted uncertainties were given by extending the propagation of the calibration uncertainty from Equation (13), giving $\sigma_{\ln[T(z)-T_R]} = \sigma_T / (T - T_R)$, and thus the corrected values with uncertainties:

$$\begin{aligned} \beta_{\text{unif}} &= 2.8216 \pm 0.0264 \text{ m}^{-1} \\ T_{L,\text{unif}} &= 113.99 \pm 0.88 \text{ }^\circ\text{C} \\ \beta_{\text{non-unif}} &= 2.8042 \pm 0.0261 \text{ m}^{-1} \\ T_{L,\text{non-unif}} &= 115.21 \pm 0.90 \text{ }^\circ\text{C} \end{aligned}$$

3.3. Steady-State Temperature Profile

The full time evolution of the distribution for both the uniform and non-uniform runs is shown in Figure 4. In the uniform run, all sixteen thermocouples relax monotonically from room temperature toward their steady-state temperature values. In the non-uniform run, the midpoints that had been locally pre-heated show a temperature peak within the first ~ -30 minutes before decaying toward the same general profile shape as the uniform run and eventually relaxing again to their steady-state temperature values.

Both plots appear visually stationary for $t \gtrsim 45$ mins. At the end of the recorded run ($t = 74$ mins) the thermocouple nearest the furnace (TCouple 1) reads 105.76 °C for the uniform run and 125.13 °C for the non-uniform run.

3.4. Entropy Transfer and Production

Figure 5 shows $d_i S/dt$ and $-d_e S/dt$ as a function of time for both the uniform and non-uniform runs, computed from Equations (11) and (12). In both cases the condition of $d_i S/dt \geq 0$ as per the second law of thermodynamics was satisfied at every point. $d_i S/dt$ decreased monotonically from its initial value to a non-zero minimum corresponding to the steady state:

$$\begin{aligned} \left. \frac{d_i S}{dt} \right|_{\text{unif}} &: 1.1395 \rightarrow 0.2047 \\ \left. \frac{d_i S}{dt} \right|_{\text{non-unif}} &: 1.0227 \rightarrow 0.2148 \end{aligned}$$

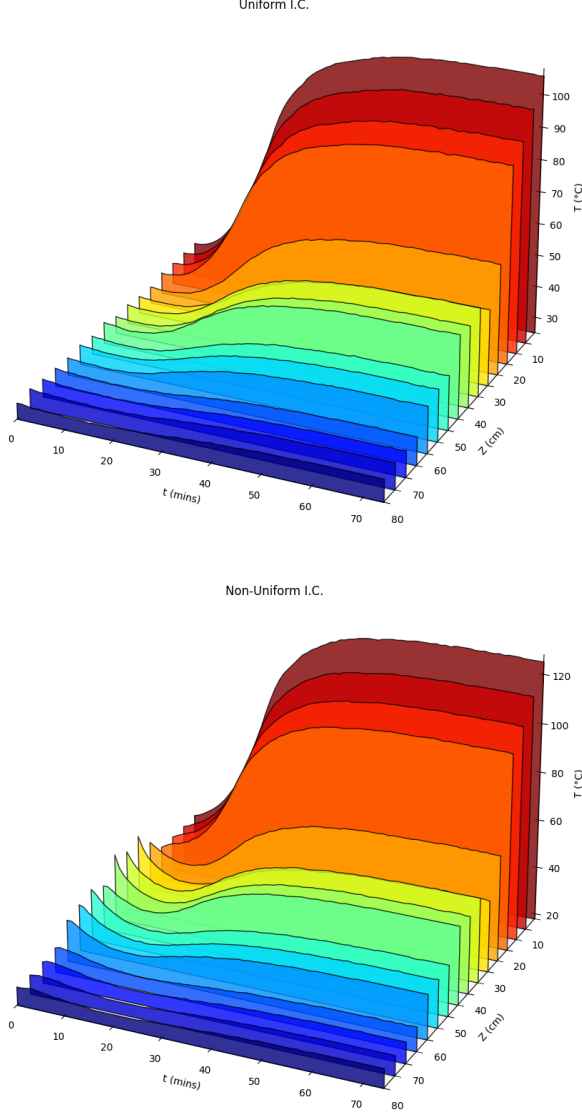


Figure 4: Steady-state plot for uniform (above) and non-uniform initial conditions (below).

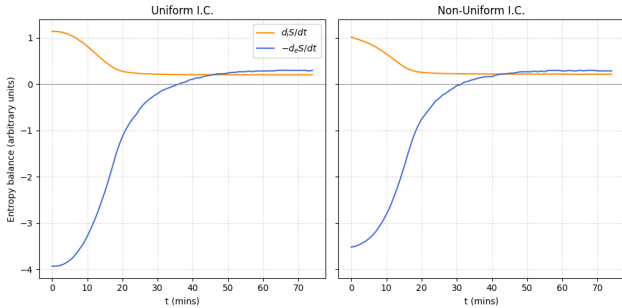


Figure 5: Entropy production and transfer rates for uniform and non-uniform initial conditions.

$-d_eS/dt$ begins negative in both runs and increases over time, approaching the corresponding d_iS/dt curve as t increases and settling slightly

above it by the end of the run at steady-state:

$$\left. \frac{-d_eS}{dt} \right|_{unif} : -3.9282 \rightarrow 0.2992$$

$$\left. \frac{-d_eS}{dt} \right|_{non-unif} : -3.8876 \rightarrow 1.1890$$

Since d_iS/dt and d_eS/dt are nonlinear functions for a large range of temperatures for many individual thermocouples, their uncertainty was obtained with Monte Carlo propagation rather than analytic. Every input alongside their uncertainties ($T_i(t)$, T_L , T_{room} , and β) were resampled 500 times from a Gaussian distribution centered on its measured or fitted values. The resulting mean and standard deviation of the resulting entropy rates gave:

$$\left. \frac{d_iS}{dt} \right|_u : 1.142 \pm 0.032 \rightarrow 0.2086 \pm 0.0091$$

$$\left. \frac{d_iS}{dt} \right|_{non-u} : 1.025 \pm 0.027 \rightarrow 0.2186 \pm 0.009$$

$$\left. \frac{-d_eS}{dt} \right|_u : -3.919 \pm 0.056 \rightarrow 0.3329 \pm 0.093$$

$$\left. \frac{-d_eS}{dt} \right|_{non-u} : -3.504 \pm 0.052 \rightarrow 0.3114 \pm 0.094$$

4. Analysis and Discussion

The fitted decay constant β showed the strongest agreement with theory discussed in Section 1.1. Because it depends solely on the material and geometric properties of the rod and its surroundings ($\beta = \sqrt{2H/\lambda r}$), it is expected to be mostly independent of the initial conditions. This independence was observed in Section 3.2: the values obtained from the fitted slopes $\beta_{unif} = 2.8216 \pm 0.0264 \text{ m}^{-1}$ and $\beta_{non-unif} = 2.8042 \pm 0.0261 \text{ m}^{-1}$, agree within their combined uncertainties ($\Delta\beta = 0.0174 \text{ m}^{-1}$ vs. $\sigma_{\Delta\beta} = 0.0371 \text{ m}^{-1}$). This confirmed that despite differing initial heating conditions, both experimental runs converged to nearly identical values of a parameter only dependent on the apparatus itself.

The heater temperature T_L also showed agreement: $113.99 \pm 0.88 \text{ }^\circ\text{C}$ (uniform) and $115.21 \pm 0.90 \text{ }^\circ\text{C}$ (non-uniform) and a difference in calculated T_L of $\Delta T_L = 1.22 \text{ }^\circ\text{C}$ within a combined uncertainty of $\sigma_{\Delta T_L} = 1.259 \text{ }^\circ\text{C}$. The main divergence happened in readings from the thermocouple closest to the heater (TCouple 1), which read

105.76 °C (uniform) and 125.13 °C (non-uniform) with a difference of 19.37 °C.

Since β remained mostly unaffected despite the large temperature shift, it is unlikely that the discrepancy came from the rod material or geometric behaviours. Instead it is more likely that they are due to local differences in the heater-rod junction between both runs, such as residual heat in the furnace or incomplete cooling of the rod before the non-uniform run due to lab time constraints. However, this discrepancy shows the advantage of extrapolating T_L from the full linear fit as it considers the entire profile rather than reading from a single near-furnace sensor.

The entropy results from Section 3.4 directly verified the theory discussed in Sections 1.2 and 1.3. The condition $d_i S/dt \geq 0$ was satisfied across all measurement points, confirming the second law of thermodynamics (Equation (7)). For both initial conditions, $d_i S/dt$ decreased monotonically from its initial value toward a non-zero steady-state minimum, supporting Prigogine’s principle (Section 1.3). Specifically, $d_i S/dt$ settled at 0.2086 ± 0.0091 (uniform) and 0.2186 ± 0.009 (non-uniform), both bounded away from zero. These results agree within their combined uncertainty ($\sigma_{\Delta d_i S} = 0.0128$), demonstrating that the steady-state is determined by boundary conditions rather than initial states.

Discrepancies arise when analysing $-d_e S/dt$. The theoretical steady-state requires $-d_e S/dt \rightarrow d_i S/dt$ exactly, but in both runs $-d_e S/dt$ instead overshoots. It rises strongly from the negative values and settles slightly above $d_i S/dt$: 0.3329 ± 0.093 versus $d_i S/dt = 0.2086 \pm 0.0091$ (uniform), and 0.3114 ± 0.094 versus $d_i S/dt = 0.2186 \pm 0.009$ (non-uniform). For the uniform run, this excess is outside the Monte Carlo uncertainty, and for the non-uniform run the curves almost overlap.

Further sources of error unquantified in β , T_L , and entropy rates likely contributed to residual discrepancies. Slight thermocouple movement in between the readings and frayed sensor may have not allowed them to seat at the same depth, introducing small spatial uncertainties not captured in β nor T_L . Uneven application of thermal paste may have also introduced minor local thermal resistances or unbalanced readings uncorrected by the beaker-water calibration. Finally, as noted beforehand, unequal initial thermal states of the rod and furnace introduced a systematic offset

between datasets, which could only be partially fixed by extrapolating the effective furnace temperature.

These error show clear possibilities for possible improvements in future iterations of this experiment. Allowing the heater and rod to return completely to room temperature between runs would eliminate the initial value thermal discrepancies. Extending run durations beyond 75 minutes would ensure the system reaches true steady-state. Although constrained by the setup in the UCD laboratory, the addition of thermocouples particularly near the furnace where the temperature gradient is steepest (Figure 4), would significantly improve samples. Finally, conducting multiple trials across different heater temperatures would improve the reliability and better validate the theory behind it.

5. Conclusion

This experiment used an aluminium rod subjected to a fixed furnace temperature and lateral Newtonian heat losses to investigate the approach to nonequilibrium steady-state, and to test Prigogine’s minimum entropy production theorem that describes the balance in entropy production and transfer. Two runs with uniform and non-uniform initial heating conditions were conducted, analysed, and compared.

The steady-state temperature profile was well described by the expected exponential form in either run, and the extracted decay constant β , which is dependent on experimental geometry only, agreed between the runs within the uncertainty ($2.8216 \pm 0.0264 \text{ m}^{-1}$ and $2.8042 \pm 0.0261 \text{ m}^{-1}$). This is despite a large difference in direct readings of temperature near the furnace between runs, most likely due to insufficient cooling between sessions rather than any change in the apparatus itself. The entropy production rate $d_i S/dt$ was determined to satisfy the second law of thermodynamics at all times in both runs and decreased monotonically towards a non-zero minimum corresponding with the steady-state (0.2086 ± 0.0091 and 0.2186 ± 0.009). The entropy transfer term $-d_e S/dt$ rose from strong negative values, crossed $d_i S/dt$, and settled slightly above it by the end of the runs instead of converging onto it exactly, and was shown to be significantly more sensitive to the fitted value of β than $d_i S/dt$, indicating that the rod might not have yet reached a true steady-

state within the 75 minute measurement window, particularly near the ambient far end despite the temperature profile graph suggesting otherwise.

Overall, the results support the theoretical discussion: the steady-state profile and the entropy production and dominated by the fixed boundary conditions of the system rather than its initial state, and the second law of thermodynamics and minimum entropy production theorem both hold within experimental uncertainties. The main limitations discussed identified concrete improvements for any repeats of this experiment.

References

- [1] D. Lurié and J. Wagensberg, “Concepts of nonequilibrium thermodynamics in discrete model of heat conduction,” *American Journal of Physics*, vol. 48, no. 10, pp. 868–872, 1980.
- [2] I. Ràfols and J. Ortín, “Heat conduction in a metallic rod with newtonian losses,” *American journal of physics*, vol. 60, no. 9, pp. 846–852, 1992.
- [3] A. Díaz-Guilera, “On heat conduction in one-dimensional solids,” *American Journal of Physics*, vol. 58, no. 8, pp. 779–780, 1990.
- [4] I. Prigogine, “Time, structure, and fluctuations,” *Science*, vol. 201, no. 4358, pp. 777–785, 1978.
- [5] L. Onsager, “Reciprocal relations in irreversible processes. i.” *Physical review*, vol. 37, no. 4, p. 405, 1931.
- [6] UCD School of Physics, “National instruments usb-6008/6009,” https://physicslabs.ucd.ie/documentation/dacq/ni_usb_6000/, 2026, accessed: 2026-09-20.

Appendix

Tables

Table 1: Slopes and intercepts and respective uncertainties for the calibration plot for Section 3.1.

TCouple	slope m (C/V)	σ_m slope (C/V)	intercept c (C)	σ_c intercept (C)
1	31.4201	0.6199	23.5024	0.5966
2	31.6219	0.6238	23.724	0.5932
3	31.3196	0.6179	23.3561	0.5988
4	31.3226	0.6179	23.5892	0.5953
5	30.8565	0.6089	24.4498	0.5823
6	31.051	0.6126	24.4679	0.582
7	31.0862	0.6133	23.6522	0.5944
8	31.0574	0.6127	24.3245	0.5841
9	31.6508	0.6245	24.3006	0.5845
10	31.351	0.6185	24.7002	0.5785
11	31.6125	0.6237	24.1059	0.5875
12	31.4322	0.6201	24.6543	0.5792
13	31.0608	0.6128	23.9839	0.5893
14	31.0607	0.6128	24.0076	0.5889
15	31.2542	0.6165	23.4353	0.5976
16	30.7427	0.6064	24.1047	0.5874

Table 2: Clean summary table for the β , T_L , and entropy values.

Quantity	Uniform	Non-uniform
β (m^{-1})	2.8216 ± 0.0264	2.8042 ± 0.0261
T_L ($^{\circ}\text{C}$)	113.99 ± 0.88	115.21 ± 0.90
$d_i S/dt$, t=0	1.142 ± 0.03161	1.025 ± 0.02708
$d_i S/dt$, steady state	0.2086 ± 0.009139	0.2186 ± 0.009331
$-d_e S/dt$, t=0	-3.919 ± 0.05582	-3.504 ± 0.05173
$-d_e S/dt$, steady state	0.3329 ± 0.09272	0.3114 ± 0.09395

CoolNewton_Analysis

September 22, 2026

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
from matplotlib.collections import PolyCollection
import matplotlib.colors as mcolors
import matplotlib.ticker as ticker

plt.rcParams['figure.figsize'] = [10, 6]
```

1 Calibration + Conversion (from CoolNewton_Data.ipynb)

1.1 Calibration Values

```
[2]: Troom = 20.2 # water taken from the tap
Vroom = np.array([-0.07552724, -0.08564685, -0.07672076, -0.08612363, -0.
↪10711619, -0.1078814,
-0.08697812, -0.1078605, -0.10617525, -0.11628798, -0.09717631, -0.11676893,
-0.09690153, -0.0976667, -0.08697812, -0.1078605 ])
```

```
[3]: Thot = 76.5 # initial hot water temperature (from boiled kettle)
Vhot = np.array([1.68162501, 1.66089737, 1.69268489, 1.68108879, 1.67023401, 1.
↪66947625,
1.68046395, 1.66964336, 1.630545, 1.64046995, 1.641546, 1.64022838,
1.68044866, 1.67969095, 1.68046395, 1.69007444])

# all subsequent Thot(n) values are of the cooling hot water, about 10C between
```

```
[4]: Thot1 = 66.3
Vhot1 = np.array([1.37514497, 1.36469981, 1.37562376, 1.38485087, 1.38422363, 1.
↪36303528,
1.39440396, 1.36317718, 1.34449696, 1.34427239, 1.35516821, 1.33377536,
1.38422363, 1.38346468, 1.39440396, 1.3938238 ])
```

```
[5]: Thot2 = 56.3
Vhot2 = np.array([1.04823292, 1.02764742, 1.05856264, 1.03753745, 1.03692532, 1.
↪02595021,
```

```
1.05726183, 1.03627992, 1.02780092, 1.01743371, 1.0278793, 1.01710724,
1.03692532, 1.0361649, 1.05726183, 1.04649546])
```

```
[6]: Thot3 = 45.8
Vhot3 = np.array([0.70088887, 0.69059502, 0.71081817, 0.70043912, 0.67941234, 0.
↪67865043,
0.70990328, 0.68895158, 0.67024086, 0.6599539, 0.68013484, 0.66979382,
0.69984166, 0.69907983, 0.70990328, 0.70938266])
```

```
[7]: Thot4 = 36.5
Vhot4 = np.array([0.40462483, 0.40461117, 0.42444038, 0.41441631, 0.38318731, 0.
↪38242416,
0.41362686, 0.39270094, 0.38419282, 0.37397005, 0.38352927, 0.3735559,
0.39340197, 0.39263886, 0.41362686, 0.39270094])
```

```
[8]: Thot5 = 26.7 # about room temperature, achieved quicker by adding ice/cold,
↪water and stirring
Vhot5 = np.array([0.07771278, 0.06755878, 0.07669592, 0.07731798, 0.04610365, 0.
↪04533909,
0.06626831, 0.0453726, 0.04706477, 0.03691765, 0.05624036, 0.03645758,
0.06653297, 0.06576849, 0.08670116, 0.06580368])
```

1.2 Calibration

```
[100]: temps=np.array([Troom, Thot, Thot1, Thot2, Thot3, Thot4, Thot5])
volts=np.stack([Vroom, Vhot, Vhot1, Vhot2, Vhot3, Vhot4, Vhot5]) # need to
↪stack the (7,16) matrices

Terr=1 # thermometer uncertainty from spec sheet

def linearfit(m,x,c):
    return m*x+c

slopes=np.zeros(16)
intercs=np.zeros(16)
slopes_err=np.zeros(16)
intercs_err=np.zeros(16)
mccov_TC=np.zeros(16)

for i in range(16):
    vs=volts[:,i]
    popt, pcov = curve_fit(linearfit, vs, temps, sigma=np.full_like(temps,
↪Terr), absolute_sigma=True)
    slopes[i], intercs[i]=popt
    slopes_err[i], intercs_err[i]=np.sqrt(np.diag(pcov))
    mccov_TC[i]=pcov[0,1]
```

```
[108]: def calib_plot(volts, temps, slopes, intercs, vzoom=None, tzoom=None,
↳ temperr=1, volterr=0.005):

    colors=plt.cm.jet_r(np.linspace(0,1,16))

    for i in range(16):
        vs=volts[:,i]

        plt.
↳ errorbar(vs,temps,yerr=temperr,xerr=volterr,fmt='o',color=colors[i],markersize=5,alpha=0.
↳ 7,capsize=5)

        vline=np.linspace(volts.min(),volts.max(),50)
        tline=slopes[i]*vline+intercs[i]
        plt.plot(vline,tline,'-',color=colors[i],label=f'TCouple {i+1}',alpha=0.
↳ 7)

        plt.xlabel('Raw Voltage (V)')
        plt.ylabel('Reference Temperatures (C)')
        plt.grid(True, linestyle='--', alpha=0.4)

        plt.legend(ncol=2, framealpha=0.5, loc='upper left')

        if vzoom is not None:
            plt.xlim(vzoom)
        if tzoom is not None:
            plt.ylim(tzoom)

        plt.tight_layout()
        plt.show()

[109]: calib_plot(volts, temps, slopes, intercs, vzoom=None, tzoom=None, temperr=1,
↳ volterr=0.005)
```

1.2.1 Conversion

```
[15]: def volt_to_C(v):
        temp_C=(v*slopes)+intercs
        return temp_C

def volt_to_K(v):
        temp_C=(v*slopes)+intercs
        return temp_C + 273.15

[16]: # error propagation for the calibrationg gave a pcov matrix, in which the
        ↪ diagonal elements are the variances of the slope and intercept, and the
        ↪ off-diagonal elements are the covariances between slope and intercept (all
        ↪ need to be considered)
        # the error propagation formula for a linear function  $y = mx + c$  is given by:
        #  $\sigma^2_T = V^2 * \sigma^2_m + \sigma^2_c + 2*V*\sigma_{mc} + m^2 * \sigma^2_V$ 
        #  $\sigma_{mc}$  is the covariance between slope and intercept, and it's the same in
        ↪ the off-diagonal, hence *2

def volt_to_C_err(v, v_err=0.0):
        temp_C=(v*slopes)+intercs
        errprop_T=(v**2)*(slopes_err**2)+(intercs_err**2) +
        ↪ 2*v*mccov_TC+(slopes**2)*(v_err**2)
        return temp_C, np.sqrt(errprop_T)

def volt_to_K_err(v, v_err=0.0):
        temp_C, err_T=volt_to_C_err(v, v_err)
        return temp_C+273.15, err_T
```

2 Analysis

2.1 Conversion

```
[17]: tc_z=0.05 # (m)
        n = 16

        terr=1
        zerr=0.005

[18]: def load_run(filename, interval_sec=60, v_err=0.0):
        data=pd.read_csv(filename)
        V=data.values
        t_C=np.round(volt_to_C(V), 2)
        t_K=np.round(volt_to_K(V), 2)
        _, t_Cerr=volt_to_C_err(V, v_err)
```



```

t_Cerr=np.round(t_Cerr, 2)
t_mins=np.arange(75) * (interval_sec / 60)

tcouple_col=[f'TCouple {i+1}' for i in range(n)]

df_C=pd.DataFrame(t_C, columns=tcouple_col, index=t_mins)
df_K=pd.DataFrame(t_K, columns=tcouple_col, index=t_mins)
df_Cerr=pd.DataFrame(t_Cerr,columns=tcouple_col, index=t_mins)

df=pd.concat([df_C, df_K, df_Cerr], axis=1, keys=['°C', 'K', ' (°C)'])
df.index.name='Time (mins)'

return df

```

```

[19]: uni_df=load_run('/Users/JoanaUCD/Library/CloudStorage/
↳OneDrive-UniversityCollegeDublin/Labs/labs-files/labs_code/Labs-Code/
↳labcode_s4/uniform_run.csv')
print(uni_df)
uni_df.to_csv('Uniform Run Data.csv')

```

	°C						
	TCouple 1	TCouple 2	TCouple 3	TCouple 4	TCouple 5	TCouple 6	
Time (mins)							
0.0	32.05	32.01	31.68	31.70	30.24	30.41	
1.0	32.05	32.01	31.92	31.46	30.24	30.16	
2.0	32.29	32.26	31.92	31.70	29.99	30.16	
3.0	32.77	32.50	32.16	31.94	29.99	29.91	
4.0	33.49	32.98	32.63	32.18	29.99	29.91	
...	...	...	...	...	...	...	
70.0	105.76	98.16	91.16	86.88	59.71	48.91	
71.0	105.76	98.41	91.40	87.12	59.96	48.91	
72.0	105.76	98.41	91.40	87.12	59.96	48.91	
73.0	105.52	98.16	91.40	86.88	59.71	48.91	
74.0	105.76	98.41	91.40	87.12	59.71	48.91	

					... (°C)		
	TCouple 7	TCouple 8	TCouple 9	TCouple 10	TCouple 7	TCouple 8	
Time (mins)							
0.0	30.96	30.42	31.80	30.75	...	0.62	0.62
1.0	30.72	30.17	31.55	30.50	...	0.62	0.63
2.0	30.48	30.17	31.55	30.50	...	0.63	0.63
3.0	30.48	29.92	31.31	30.24	...	0.63	0.63
4.0	30.48	29.92	31.06	30.24	...	0.63	0.63
...	...	...	...	...	...	...	
70.0	52.71	50.55	50.92	41.85	...	0.40	0.40
71.0	52.71	50.55	51.17	42.10	...	0.40	0.40
72.0	52.71	50.55	51.17	41.85	...	0.40	0.40
73.0	52.71	50.55	51.17	41.85	...	0.40	0.40

72.0	0.56	0.56
73.0	0.56	0.56
74.0	0.56	0.56

[75 rows x 48 columns]

2.2 Effective Oven Temp

```
[ ]: # have [T(z)-T_R] = (T_L-T_R) * exp(-beta * z) from the papers
# take natural log of both sides to get ln([T(z)-T_R]) = ln(T_L-T_R) - beta * z
    and plot

T_SS_uni=uni_df['°C'].iloc[-1] # steady state temperature for uniform run
z_pos=np.linspace(0.05, 0.8, 16)
yuni_semielog=np.log(T_SS_uni - Troom)
unislope, uniinterc=np.polyfit(z_pos,yuni_semielog,1)
unibeta=-unislope
T_L_uni=Troom+np.exp(uniinterc)

T_SS_nonuni=nonuni_df['°C'].iloc[-1] # steady state temperature for non-uniform
    run
ynonuni_semielog=np.log(T_SS_nonuni - Troom)
nonunislope, nonuniinterc=np.polyfit(z_pos,ynonuni_semielog,1)
nonunibeta=-nonunislope
T_L_nonuni=Troom+np.exp(nonuniinterc)
```

```
[22]: errT_uni=uni_df[' (°C)'].iloc[-1]
errlnT_uni=errT_uni/(T_SS_uni-Troom)

print(f'Uniform run beta = {unibeta:.4f} 1/m')
print(f'Uniform run T_L = {T_L_uni:.2f} C')

errT_nonuni=nonuni_df[' (°C)'].iloc[-1]
errlnT_nonuni=errT_nonuni/(T_SS_nonuni-Troom)

print(f'Non-uniform run beta = {nonunibeta:.4f} 1/m')
print(f'Non-uniform run T_L = {T_L_nonuni:.2f} C')
```

```
Uniform run beta = 2.7458 1/m
Uniform run T_L = 113.99 C
Non-uniform run beta = 2.7656 1/m
Non-uniform run T_L = 116.39 C
```

```
[23]: fig, ax=plt.subplots(figsize=(10,6))

ax.errorbar(z_pos, yuni_semielog, xerr=zerr, yerr=errlnT_uni ,
    fmt='o',label='Uniform Data', color='blueviolet',alpha=0.7,capsize=5)
```

```

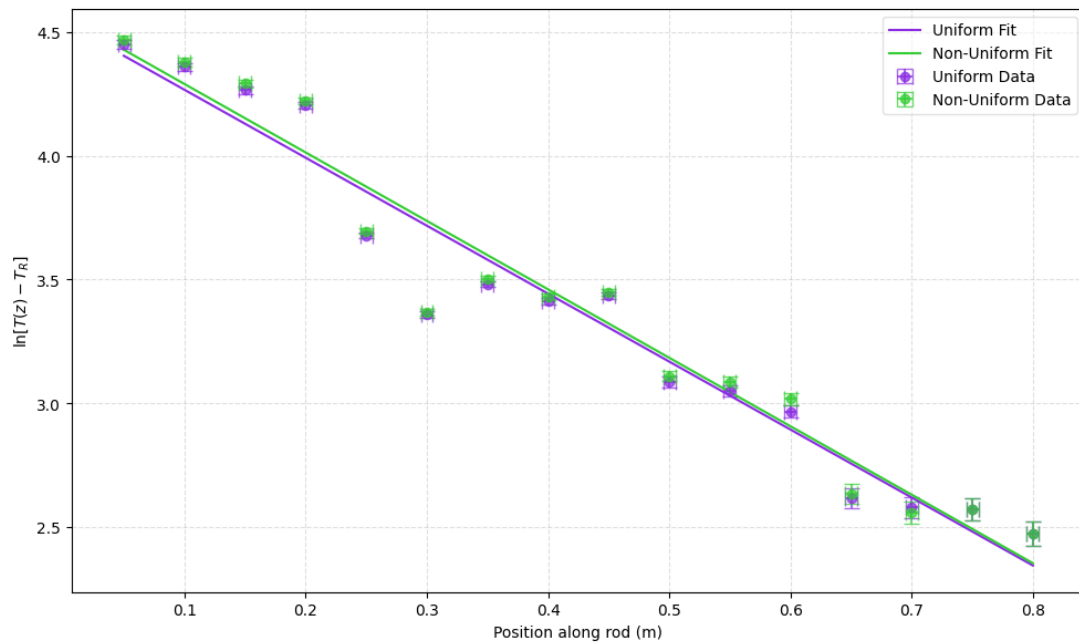
ax.errorbar(z_pos, ynonuni_semi-log, xerr=zerr, yerr=errlnT_nonuni,
    ↪fmt='o',label='Non-Uniform Data', color='limegreen',alpha=0.7,capsize=5)
zcont=np.linspace(0.05,0.8,50)

ax.plot(zcont, (unislope*zcont+uniinterc), '-', label='Uniform Fit',
    ↪color='blueviolet')
ax.plot(zcont, (nonunislope*zcont+nonuniinterc), '-', label='Non-Uniform Fit',
    ↪color='limegreen')
ax.set_xlabel('Position along rod (m)')
ax.set_ylabel(r'$\ln[T(z)-T_R]$')

ax.legend(framealpha=0.6)
ax.grid(True, linestyle='--', alpha=0.4)

plt.tight_layout()
plt.show()

```



```

[114]: def explogpart(z,beta,lnA):
        return -beta*z + lnA

popt_uni, pcov_uni = curve_fit(explogpart,z_pos,yuni_semi-log, sigma=errlnT_uni,
    ↪absolute_sigma=True)
unibeta_w, uniinterc_w =popt_uni # slope gives beta value
unc_unibeta, unc_uniinterc =np.sqrt(np.diag(pcov_uni))

```

```

unc_uniTL = np.exp(uniinterc_w) * unc_uniinterc # error propagation from T_L =
↳ T_R + exp(lnA)

popt_nonuni, pcov_nonuni = curve_fit(explogpart, z_pos, ynonuni_semiplot,
↳ sigma=errlnT_nonuni, absolute_sigma=True)
nonunibeta_w, nonuniinterc_w = popt_nonuni
unc_nonunibeta, unc_nonuniinterc = np.sqrt(np.diag(pcov_nonuni))
unc_nonuniTL = np.exp(nonuniinterc_w) * unc_nonuniinterc

print(f'UNIFORM:\nbeta= {unibeta_w:.4f} ± {unc_unibeta:.4f} 1/m\nT_L= {Troom+np.
↳ exp(uniinterc_w):.2f} ± {unc_uniTL:.2f} C\n')
print(f'NON-UNIFORM:\nbeta= {nonunibeta_w:.4f} ± {unc_nonunibeta:.4f} 1/m\nT_L=
↳ {Troom+np.exp(nonuniinterc_w):.2f} ± {unc_nonuniTL:.2f} C\n')

```

UNIFORM:

beta= 2.8216 ± 0.0264 1/m

T_L= 113.99 ± 0.88 C

NON-UNIFORM:

beta= 2.8042 ± 0.0261 1/m

T_L= 115.21 ± 0.90 C

2.3 Steady-State Profile

```

[76]: def waterfallplot_3d(t, z, T, ax=None, baseline=None, facecolor='white',
↳ edgecolor='black', xlabel='t (mins)', ylabel='Z (cm)', zlabel='T (°C)',
↳ title=None, figsize=(7,5)):

    # t: shape(nt, )
    # z: shape(nz, )
    # T: shape(nt, nz)

    t = np.asarray(t); z = np.asarray(z); T = np.asarray(T) # np.asarray used
↳ as it treats as an array without converting unless necessary
    assert T.shape == (len(t), len(z)) # ie, checks that T has this shape (nt,
↳ nz)

    if ax is None: # making a 3d plot if one hadn't been previously defined,
↳ just in case
        fig = plt.figure(figsize=figsize)
        ax = fig.add_subplot(111, projection='3d') # (1,1,1) is default, in
↳ place of (nrows, ncols, index)

    if baseline is None: # calculating the lowest point (bottom edge)
        baseline=T.min() - 0.05 * (T.max()-T.min()) # takes lowest and subtracts
↳ 5% of total range to make the graph look neater

```

```

    order=np.argsort(-z) # make it look better with the large z first and small
    ↪z last (ie, in order of closeness to oven)
    for j in order:
        curve= T[:,j]
        _2dpart=[(t[0],baseline)] + list(zip(t,curve)) + [(t[-1],baseline)] #
    ↪bottomleft corner, data (paired time t and temp curve T), bottom right corner

        cmap=plt.get_cmap('jet_r')
        norm=mcolors.Normalize(vmin=z.min(),vmax=z.max())
        colors=cmap(norm(z[j]))

        poly= PolyCollection([_2dpart], facecolor=colors, edgecolor=edgecolor,
    ↪alpha=0.8) # turns the corners into a 2d shape
        ax.add_collection3d(poly, zs=z[j], zdir='y') # turns into 3d shape,
    ↪with y-axis as the depth axis

        ax.set_xlim(t.min(), t.max())
        ax.set_ylim(z.min(), z.max())
        ax.set_zlim(baseline, T.max() * 1.02) # going 2% above limit to make it
    ↪neater

        ax.set_xlabel(xlabel, labelpad=5)
        ax.set_ylabel(ylabel,labelpad=5)
        ax.set_zlabel(zlabel,labelpad=5)
        if title:
            ax.set_title(title, y=0.96)

        ax.view_init(elev=25, azim=-65) # aesthetics (change elevation and
    ↪azimuthal to where we are "looking")
        ax.invert_yaxis() # z=0 (near source) put at back

        ax.xaxis.pane.set_alpha(0); ax.yaxis.pane.set_alpha(0); ax.yaxis.pane.
    ↪set_alpha(0) # background (0-1 value, 0=transparent)
        ax.grid(False)

        ax.yaxis.set_minor_locator(ticker.AutoMinorLocator(2))

    return ax

def waterfall_df(df,z,unit='°C', **kwargs): # **kwargs just copies any style
    ↪change
    t=df.index.values
    T=df[unit].values

    return waterfallplot_3d(t,z,T,**kwargs)

```

2.4 Entropy Production

$$2.4.1 \quad \frac{d_e S}{dt} \propto \int_0^L \frac{1}{T} \left(\frac{\partial^2 T}{\partial z^2} \right) dz - \int_0^L \frac{1}{T^2} \left(\frac{\partial T}{\partial z} \right)^2 dz - \frac{2H}{\lambda r} \int_0^L \frac{1}{T} (T - T_R) dz$$

this is the term which derives from the transfer of heat across the boundary of the system

$$2.4.2 \quad \frac{d_i S}{dt} \propto \int_0^L \frac{1}{T^2} \left(\frac{\partial T}{\partial z} \right)^2 dz$$

this is the term due to exchanges inside the system

both of these expressions use only terms experimentally accessible these then give (which can be plotted and computed) for each thermocouple:

$$2.4.3 \quad \frac{d_e S}{dt} \propto -\frac{1}{l} \frac{1}{T_0} (T_1 - T_0) + \frac{1}{l} \frac{1}{T_n} (T_n - T_{n-1}) - \beta^2 l \sum_{i=1}^{n-1} \left(1 - \frac{T_n}{T_i} \right)$$

and

$$2.4.4 \quad \frac{d_i S}{dt} \propto -\frac{1}{l} \sum_{i=0}^{n-1} (T_{i+1} - T_i) \left(\frac{1}{T_{i+1}} - \frac{1}{T_i} \right)$$

in this case, T is in Kelvin and the entropy will always be greater than or equal to 0 (as per thermodynamic law) and $dS/dt = d_i S/dt + d_e S/dt$ as the total entropy of the system exchanged with the surroundings

```
[56]: def entropies(T_K, T0, Tn, l, beta, tol=0.01, max_extra=2000):
    # T0 = boundary temp (furnace)
    # Tn = ambient bath temp

    nTC, m = T_K.shape # for the 16 thermocouples
    T0=np.broadcast_to(T0,(nTC,)).astype(float) # stretches a single value to
    ↪match a specific shape w/o duplicating it

    z_lastTC = m * l # where the last thermocouple is (calculated)
    dT0 = float(np.mean(T0))-Tn # finding average temp above ambient
    ↪(difference)

    n_extra = 0
    z=z_lastTC
    while dT0 * np.exp(-beta * z) > tol and n_extra < max_extra: # extend steps
    ↪z by length l until temp decay satisfied
        n_extra +=1
        z +=l # this moving them
    z_extra=z_lastTC +l *np.arange(1,n_extra+1)
    T_extra=Tn+dT0 *np.exp(-beta * z_extra) # virtual points calculations
    T_extra=np.tile(T_extra, (nTC,1)) # evaluating the same things for all the
    ↪virtual points
```

```

T_tot = np.column_stack([T0, T_K, T_extra, np.full(nTC, Tn)]) # makes a
↳ (nt, m+n_extra+2) shape (ie "gluing" the thermocouple columns together to
↳ make a singular array)

dT = np.diff(T_tot, axis=1) # subtracts each column from the subsequent one
↳ (temp difference between each thermocouple at each taken measurement)
dinvT = np.diff(1/T_tot, axis=1) # same but for inverse temp

diS_dt = -(1/l) * np.sum(dT * dinvT, axis=1)

deS_dt = (-(1/l) * (1/T_tot[:,0]) * (T_tot[:,1] - T_tot[:,0]) # entropy in
+ (1/l) * (1/T_tot[:, -1]) * (T_tot[:, -1] - T_tot[:, -2]) # entropy
↳ out
- beta**2 * l * np.sum(1- Tn/T_tot[:, 1:-1], axis=1)) # entropy
↳ lost

return diS_dt, deS_dt

```

```

[ ]: T0_uni = T_L_uni + 273.15
T0_nonuni = T_L_nonuni + 273.15
Tn = Troom + 273.15

diS_uni, deS_uni = entropies(uni_df['K'].values, T0_uni, Tn, tc_z, unibeta)
dS_uni = diS_uni + deS_uni

diS_nonuni, deS_nonuni = entropies(nonuni_df['K'].values, T0_nonuni, Tn, tc_z,
↳ nonunibeta)
dS_nonuni = diS_nonuni + deS_nonuni

print(f'Uniform:\ndiS/dt= {diS_uni[0]:.4f} --> {diS_uni[-1]:.4f}\n')
print(f'Uniform:\ndeS/dt= {deS_uni[0]:.4f} --> {deS_uni[-1]:.4f}\n')
print(f'Non-uniform:\ndiS/dt= {diS_nonuni[0]:.4f} --> {diS_nonuni[-1]:.4f}\n')
print(f'Non-uniform:\ndeS/dt= {deS_nonuni[0]:.4f} --> {deS_nonuni[-1]:.4f}\n')
print(f'Second thermodynamic law check:\nuniform= {np.all(deS_uni >=
↳ 0)}\nnon-uniform= {np.all(diS_nonuni >= 0)}') # just checking if theyre
↳ greater than or equal to 0

```

Uniform:
diS/dt= 1.1395 --> 0.2047

Uniform:
deS/dt= 3.9282 --> -0.2992

Non-uniform:
diS/dt= 1.2125 --> 0.3581

Non-uniform:

diS/dt= 3.8876 --> -1.1890

Second thermodynamic law check:

uniform= True

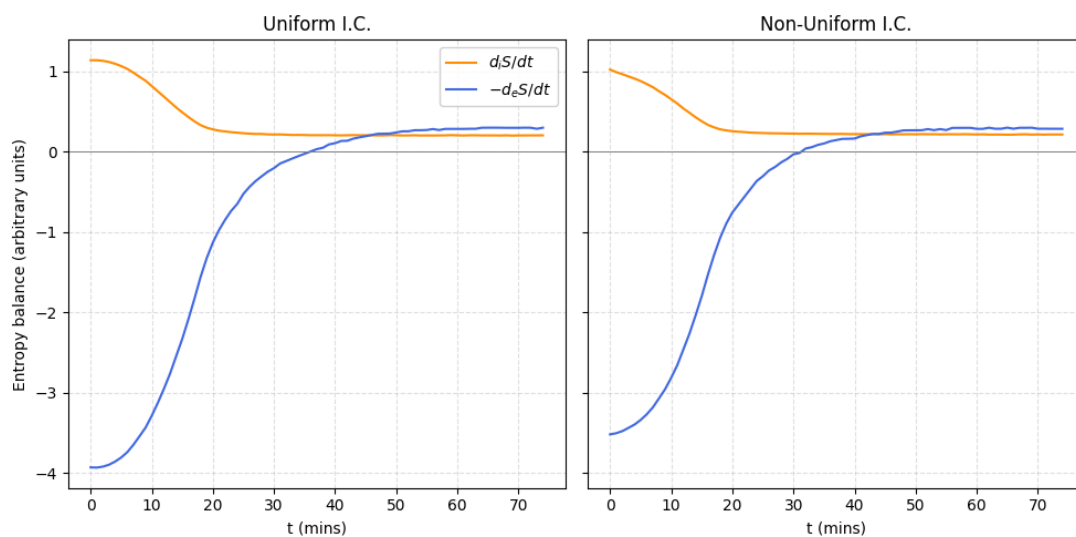
non-uniform= True

```
[113]: fig, axes=plt.subplots(1,2,figsize=(10,5), sharey=True)

for ax, t, diS, deS, dS, label in [
    (axes[0], uni_df.index.values, diS_uni, -deS_uni, dS_uni, 'Uniform I.C.'),
    (axes[1], nonuni_df.index.values, diS_nonuni, -deS_nonuni, dS_uni,
    ↪ 'Non-Uniform I.C.')
]:
    ax.plot(t, diS, '--', color='darkorange', label=r'$d_iS/dt$')
    ax.plot(t, deS, '--', color='royalblue', label=r'$-d_eS/dt$')
    ax.axhline(0, color='grey', lw=0.7)
    ax.set_xlabel('t (mins)')
    ax.set_title(label)
    ax.grid(True, linestyle='--', alpha=0.4)

axes[0].set_ylabel('Entropy balance (arbitrary units)')
axes[0].legend(framealpha=0.9)

plt.tight_layout()
plt.show()
```



```
[59]: # using monte carlo error propagation because otherwise too complicated and
    ↪ long to do the calc
```



```

# actually thinking of all that math and figuring out the analytical error
↳propagation makes me sick
# monte carlo also follows a gaussian curve of errors, which works fine because
↳we already calculated the uncertainties on individual measurements

def entropy_mcunc(df, TL, unc_TL, Troom, Terr, l, beta, unc_beta, n_trials=500,
↳seed=0):
    rng = np.random.default_rng(seed)
    T_meas=df['°C'].values
    unc_T=df[' (°C)'].values
    nTC, m =T_meas.shape

    diS_trials=np.empty((n_trials,nTC)) # empty arrays with this shape
    deS_trials=np.empty((n_trials,nTC))

    for k in range(n_trials):
        T_sample_K=(T_meas +rng.normal(0,unc_T)) +273.15 # takes the
↳experimental data and adds random noise based on the measured uncertainty
↳and converts to K
        TO_K=rng.normal(TL, unc_TL) +273.15
        Tn_K=rng.normal(Troom, Terr) +273.15
        beta_K=rng.normal(beta, unc_beta)
        diS_K, deS_K=entropies(T_sample_K, TO_K, Tn_K, l, beta_K) # gets the
↳curves for each individual version in which random noise was added
        diS_trials[k]=diS_K
        deS_trials[k]=deS_K

    return diS_trials.mean(axis=0), diS_trials.std(axis=0), deS_trials.
↳mean(axis=0), deS_trials.std(axis=0) # mean = 'true' curve, std = standard
↳deviation

```

```

[ ]: diS_uni_min, diS_uni_std, deS_uni_min, deS_uni_std = entropy_mcunc(uni_df,
↳T_L_uni, unc_uniTL, Troom, Terr, tc_z, unibeta_w, unc_unibeta)

diS_nonuni_min, diS_nonuni_std, deS_nonuni_min, deS_nonuni_std=
↳entropy_mcunc(nonuni_df, T_L_nonuni, unc_nonuniTL, Troom, Terr, tc_z,
↳nonunibeta_w, unc_nonunibeta)

print('Uniform I.C.')
print(f'd_iS/dt at t=0 (min): {diS_uni_min[0]:.4g} ± {diS_uni_std[0]:.4g}\nd_iS/
↳dt at steady state: {diS_uni_min[-1]:.4g} ± {diS_uni_std[-1]:.4g}')
print(f'd_eS/dt at t=0 (min): {deS_uni_min[0]:.4g} ± {deS_uni_std[0]:.4g}\nd_eS/
↳dt at steady state: {deS_uni_min[-1]:.4g} ± {deS_uni_std[-1]:.4g}\n')

print('Non-Uniform I.C.')

```